

Universidad Nacional Autónoma de México

Facultad de Ingeniería

División de Ingeniería Eléctrica



Carrera: Ingeniería en Computación

Profesor: Óscar Manuel Ruíz Hurtado

Curso: Sistemas Distribuidos

S11 - Bitácora de MexiBanco: Poliglota

Alumno: Rodríguez Zamora Joshua

Fecha de entrega: 07/11/2026

Semestre: 2027-1

Grupo: 02

Contenido

1. **Punto de Partida: diez consultas, diez lecturas a Postgres (sección 3.4).** 3
2. **La caché** 4
3. **Medidas** 6

Prueba A: 6

Prueba B: 7

Con caché: 7

Sin caché: 8

4. **Los avisos** 8
5. **El duplicado y el dato viejo** 13
6. **Preguntas de cierre** 15

1. **Punto de Partida: diez consultas, diez lecturas a Postgres (sección 3.4).**

En esta práctica se incorpora el uso de Redis y MongoDB, usándolos para tener una copia en la caché y una base de datos de la bitácora de avisos respectivamente. Por lo que es necesario añadirlos en el contenedor del sistema distribuido de mexi banco y una vez que se realiza esta incorporación y lógica en el código, veremos que funcionan adecuadamente de la siguiente forma:

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli ping
PONG

jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments()'
0
```

imagen 1. Redis y MongoDB funcionando.

Y generaremos cuentas con las cuales podremos hacer pruebas y que la práctica salga adelante

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000001","titular":"Ana","saldoInicial":1000}'
HTTP/1.1 409 Conflict
transfer-encoding: chunked
X-Instancia: 5163e9aabfbe
Content-Type: application/problem+json
Date: Tue, 03 Nov 2026 04:56:26 GMT

{"type":"about:blank","title":"Conflict","status":409,"detail":"Ya existe una cuenta con la CLABE 002180000000000001","instance":"/cuentas"}
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000002","titular":"Beto","saldoInicial":500}'
HTTP/1.1 409 Conflict
transfer-encoding: chunked
X-Instancia: 6c72ad300c78
Content-Type: application/problem+json
Date: Tue, 03 Nov 2026 04:56:43 GMT

{"type":"about:blank","title":"Conflict","status":409,"detail":"Ya existe una cuenta con la CLABE 002180000000000002","instance":"/cuentas"}
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000003","titular":"Carla","saldoInicial":300}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: ae913b3b554f
Content-Type: application/json
Date: Tue, 03 Nov 2026 04:57:00 GMT

{"id":13,"clabe":"002180000000000003","titular":"Carla","saldo":300}
```

imagen 2. Cuenta de Ana, Beto y Carla creadas.

Así mismo, a modo de comprobación de que el sistema funciona bien, haremos una transferencia

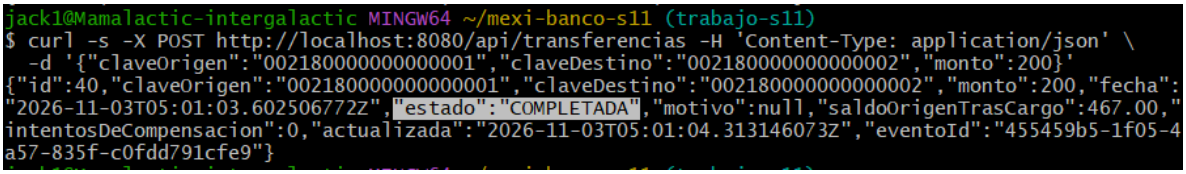


imagen 3. Transferencia exitosa.

Y ahora, con todo el entorno establecido y como dios manda, procederemos a hacer una medición con la que podremos ver como funciona actualmente nuestro sistema antes de hacer los cambios y mejoras del laboratorio de hoy

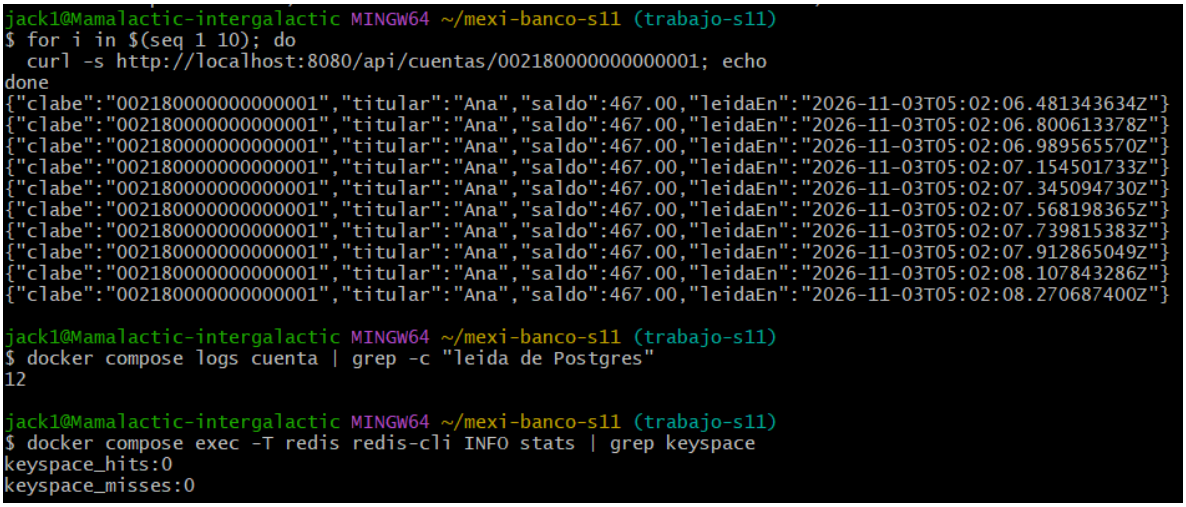


imagen 4. Mediciones del sistema "Antes".

Estos resultados muestran que hubieron 12 leídas de postgres, siendo 10 de ellas las consultas y las otras dos las lecturas de las transferencias. Mientras que se muestra que Redis no ha sido utilizado por ningún servicio con un total de 0 keyspace_hits y misses.

2. La caché

En este caso, utilizaremos un patrón de caché llamado cache-aside, en el que la app pregunta primero a la caché y, si no está, va a la base y deja una copia para la próxima vez. Mientras que cuando un saldo se cambia de cualquier forma, la copia de la caché se borra y la siguiente consulta toma este nuevo valor de referencia.

En el código afectado, colocamos librerías que permiten el uso del caché y la comunicación con redis. Además de registrar los movimientos y actualizaciones tanto con PostgreSQL y con Redis mismo. Utilizando valores de las cuentas para crear logs.

Sin embargo, se tomaron consideraciones para evitar que, al buscar un saldo, se tome en cuenta un saldo cacheado. Esto se evita al no aceptar que el método consultar pueda subir datos a la caché. De esta forma el saldo que se consulta siempre será el más actualizado y así no se aprobará ningún cargo con saldos pasados (desactualizados).

Para comprobar que el servicio está funcionando adecuadamente haremos una repetición de una consulta de saldo. De esta forma veremos si está funcionando la caché y además si lo está haciendo como esperamos.

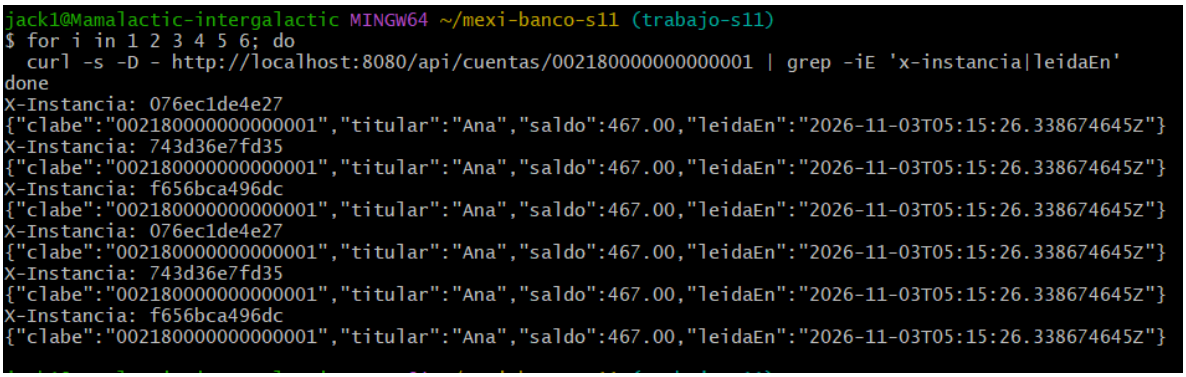


imagen 5. Consulta de saldo repetida.

Después, veremos cuantas veces es que se consultó Postgres en los últimos 30 segundos

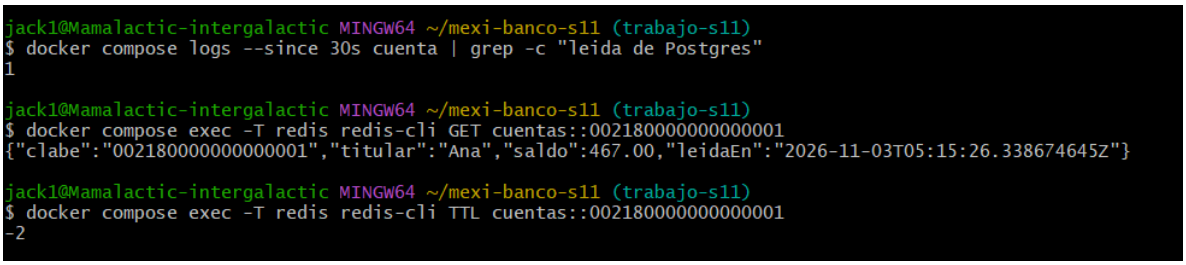


imagen 6. Lectura de postgres y actuar de la caché.

Estos resultados indican que sólo se leyó Postgres una vez y en base a ello se hizo una copia en Redis, mientras que las demás consultas leyeron este dato de Redis, además de que muestra el momento en que se leyó.

Ahora, para seguir testeando a la caché y la copia que generó a partir de las consultas repetidas de postgres. Haremos un experimento para comprobar que, en efecto, se borran los datos después de que postgres es consultado por primera vez, esto se hace desde otra terminal, con un espía de Redis (monitor)



imagen 7. Se muestra la eliminación después de las consultas.

En este caso se muestra que hay tres copias que comparten la caché (cuenta) y que cada una borra la copia del saldo al transferir.

3. Medidas

En esta sección veremos los aciertos de Redis con un Hit ratio.

Prueba A:

En este caso se hará una consulta del saldo de la cuenta de Ana para comprobar si después de hacer la copia de postgres y almacenarla en caché el rendimiento baja por tener que conectarse ahora con la caché:

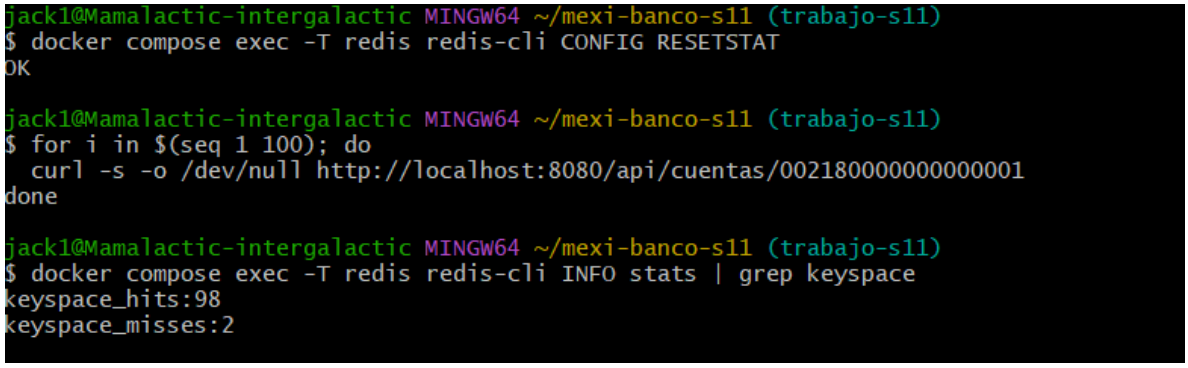


imagen 8. Pruebas de hit ratio.

En esta prueba sólo hubo 2 fallos a comparación de los 98 hits. Lo que indica que la caché no crea problemas mayores en este caso.

Prueba B:

Con la prueba B buscamos verificar si los múltiples movimientos pueden alterar la base de datos significativamente como para que la caché y postgres se empiecen a confundir respecto a los datos que borran y como leen el siguiente dato.

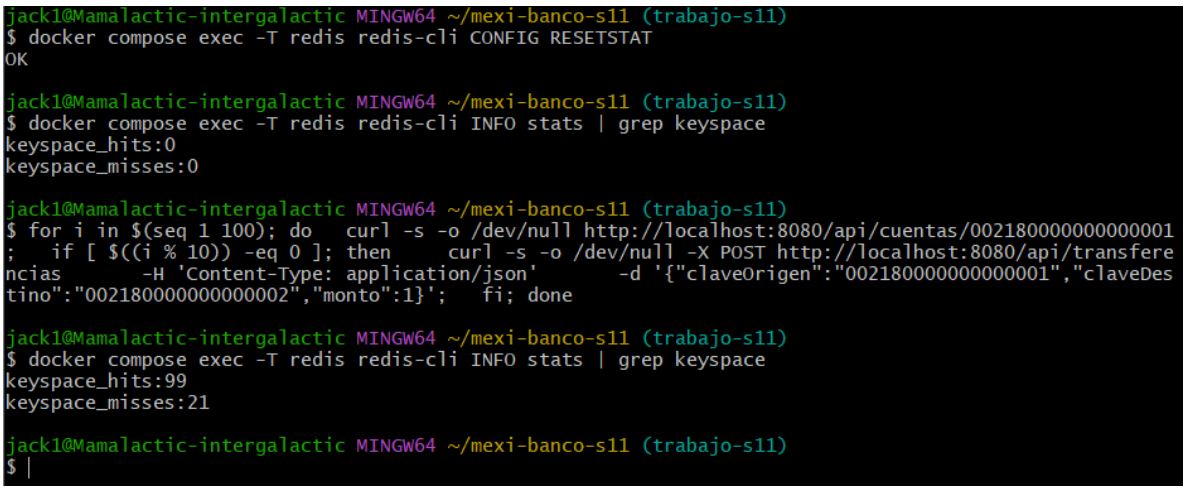


imagen 9. Prueba B exitosa.

Cada transferencia borra a Ana y a Beto, y la siguiente lectura de cada uno falla. Lo que se puede apreciar en la cantidad de misses y hits. Con esto tomamos mediciones respecto a cómo se generan ciertos procesos en el sistema que tenemos actualmente.

Ahora, la siguiente consideración es medir como funciona la caché respecto a la latencia, viendo que tanto nos ayuda la implementación de este tipo de memoria al procesamiento de peticiones:

Con caché:

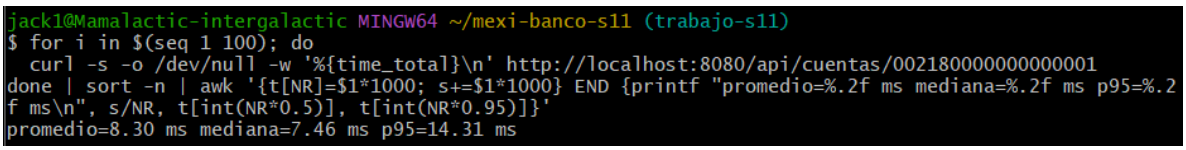


imagen 10. Mediciones de 100 consultas con caché.

Encontramos un tiempo bajo, en el orden de los milisegundos, con un promedio y una mediana ligeramente similar.

Sin caché:

Ahora, la prueba sin caché:

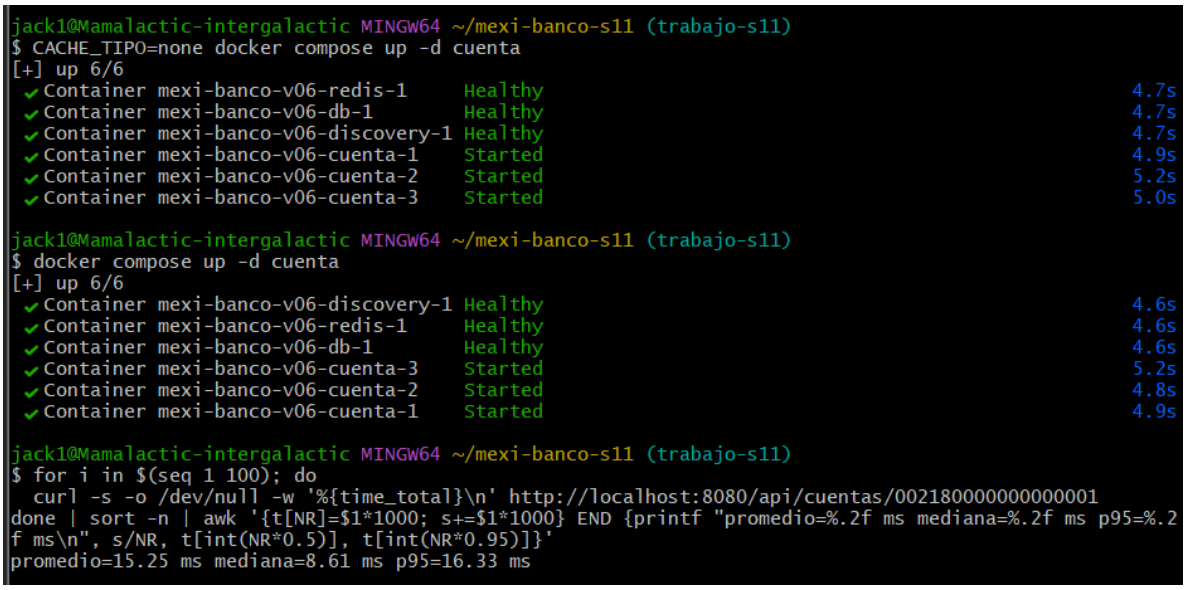


imagen 11. Mediciones del sistema sin caché.

En esta prueba notamos un incremento de tiempo en todas las mediciones, destacándose en su mayoría el tiempo en promedio que toma cada consulta. Lo que nos indica que la caché sí hace un cambio significativo en cuanto al servicio en general. Usa los recursos de mejor forma y permite que el sistema no gaste recursos en la base de datos y la consulta de datos en ella.

Definitivamente mostrando una optimización con la caché

4. Los avisos

Aquí buscamos hacer más completo al servicio, implementando un aviso en tres formas diferentes al hacer una transferencia. Permitiéndole al usuario un poco de retroalimentación del sistema y así avisarle de cualquier movimiento que implique el uso de su cuenta.

En este caso, se modifican las clases de BitacoraAvisos dentro del sistema de notificaciones, agregando las plantillas de cada mensaje, ya sea de texto (SMS), correo o Push.

Después de generar los cambios correspondientes tanto en código como en el volumen de Docker (actualizar los contenedores con los cambios más recientes). Hacemos las pruebas del servicio de avisos con movimientos entre cuentas.

Haremos transferencias de Ana a Beto y luego de Ana a Carla, sólo para después entrar a los logs del sistema y comprobar cómo se ven los avisos de estos movimientos en cada usuario.

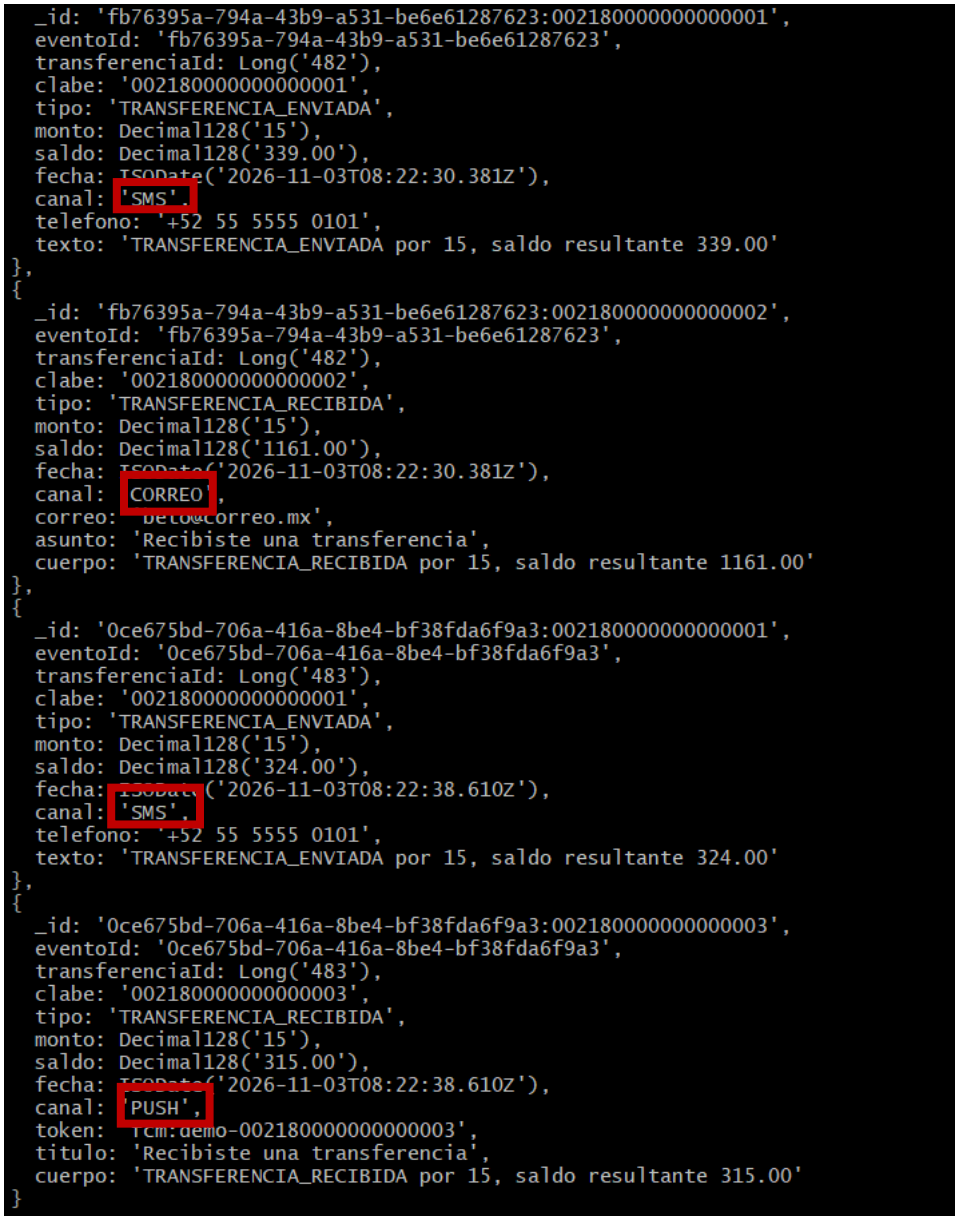


imagen 12. Avisos a cada usuario implicado.

Se muestra en la imagen anterior que se levanta un aviso a cada usuario que estuvo implicado en una transferencia, en este caso, Ana tiene dos avisos por SMS por realizar dos transferencias diferentes mientras que Beto tiene un aviso por correo y Carla un aviso de tipo PUSH.

Ahora, revisando la bitácora de Ana únicamente vemos

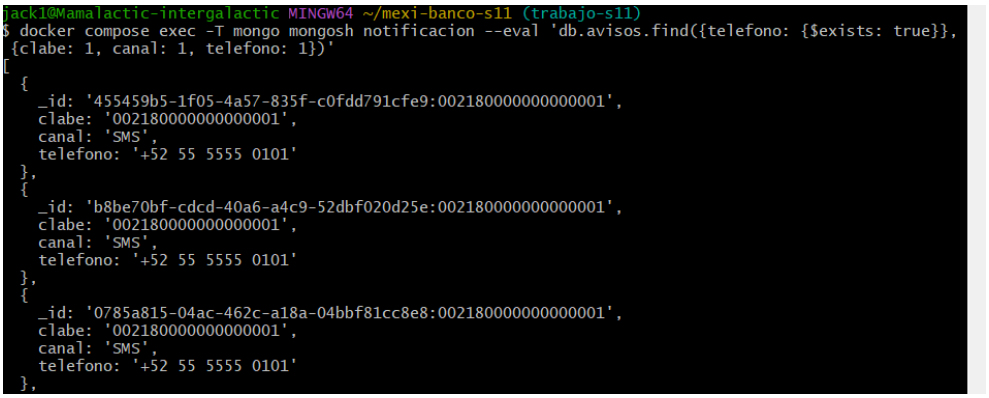


imagen 13. últimos avisos registrados para ANA.

Ahora, veremos específicamente una bitácora de Beto, al realizar 12 transferencias de Ana a Beto por el valor de 1 peso, comprobaremos como se visualiza de forma aislada el registro en la bitácora de Beto:

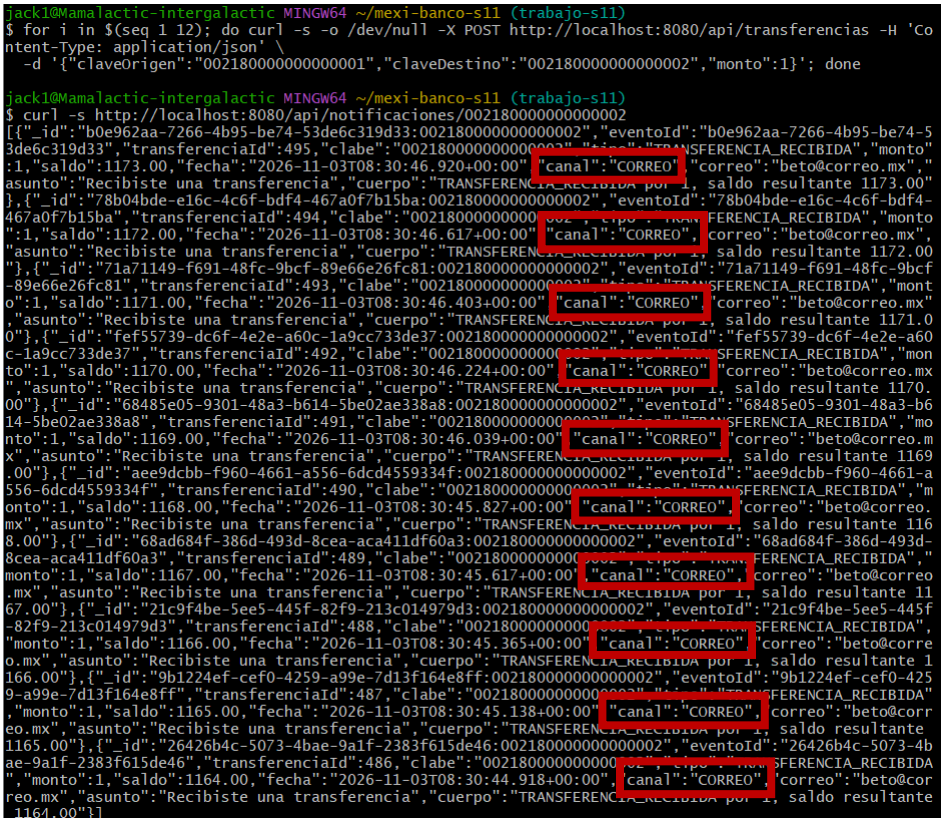


imagen 14. 12 transferencias recibidas.

Todas estas aparecen en su correo, como se señala.

En este caso sólo se buscan las ultimas 10 consultas que quedaron en la etapa IXSCAN.



imagen 15. Consultas en IXSCAN.

Ahora, se hace una búsqueda general, revisando todos los archivos de transferencias.

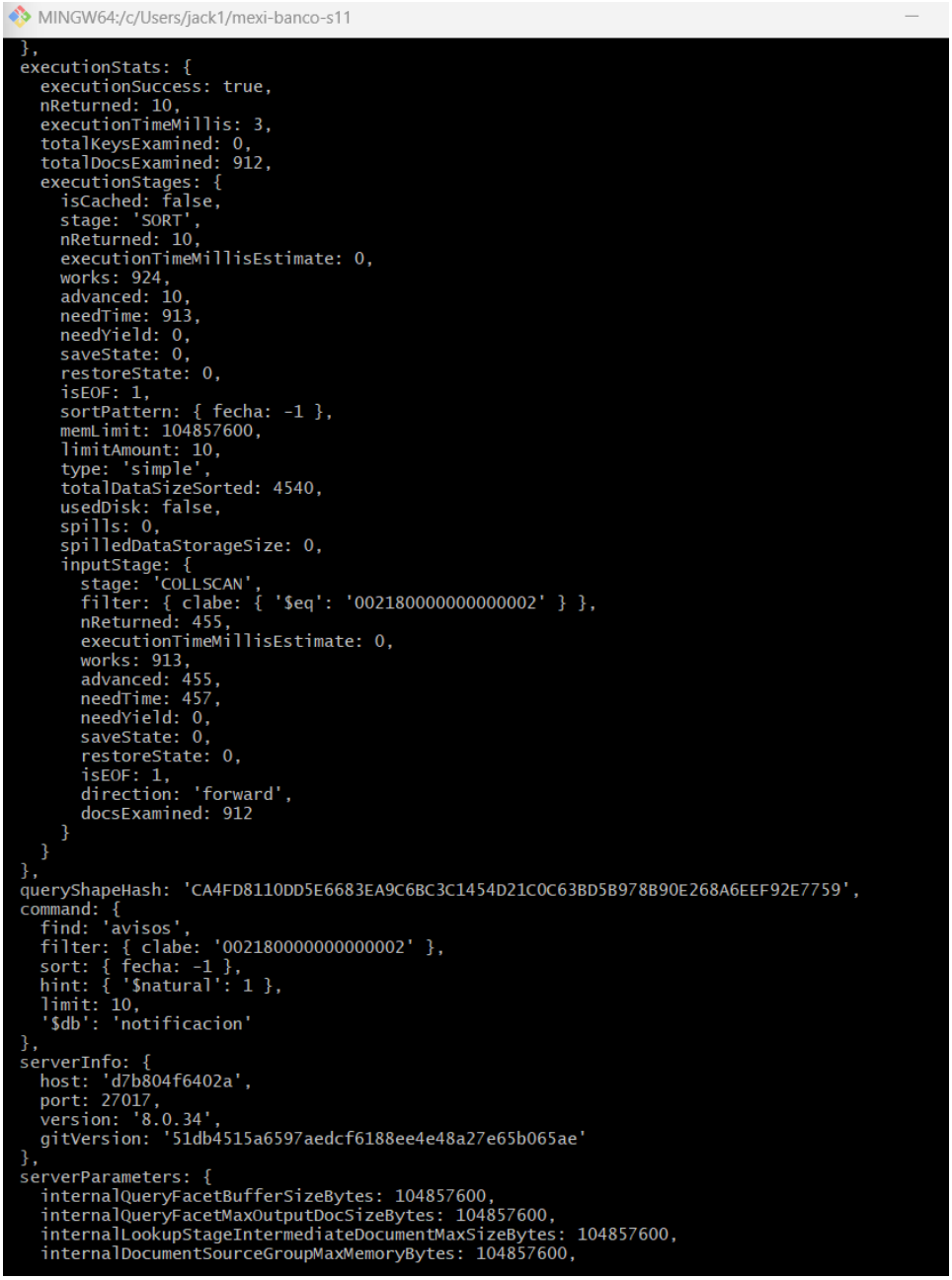


imagen 16. Registros generales.

5. El duplicado y el dato viejo

Verificaremos ahora que sucede cuando la mensajería detecta un evento duplicado, primero se provoca el evento desde la API de RabbitMQ con CLABES inventadas.

Lo que podemos apreciar aquí es que, al hacer el duplicado, el log de notificaciones identifica que se repitió un evento y rechaza los eventos duplicados tomándolos como que ya estaban registrados.

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ curl -u mexibanco:mexibanco -X POST http://localhost:15672/api/exchanges/%2F/transferencias/publis
h \
  -H 'Content-Type: application/json' \
  -d '{"properties":{"delivery_mode":2,"content_type":"application/json"},"routing_key":"transferenc
ia.realizada","payload_encoding":"string","payload":"{\"eventoId\":\"11111111-2222-4333-8444-5555555
55555\",\"tipo\":\"TransferenciaRealizada\",\"transferenciaId\":9001,\"clabeOrigen\":\"0021800000000
00091\",\"clabeDestino\":\"002180000000000092\",\"monto\":25,\"saldoOrigen\":475,\"saldoDestino\":52
5,\"fecha\":\"2026-10-31T13:00:00Z\"}"}'
{"routed":true}
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments({eventoId: "111
11111-2222-4333-8444-555555555555"})'
2

jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose logs notificacion | grep "ya estaba"
notificacion-1  | 2026-11-04T05:21:13.151Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos
.BitacoraDeAvisos            : [avisos] 11111111-2222-4333-8444-555555555555:002180000000000091 ya e
staba: evento repetido, no se duplica
notificacion-1  | 2026-11-04T05:21:13.154Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos
.BitacoraDeAvisos            : [avisos] 11111111-2222-4333-8444-555555555555:002180000000000092 ya e
staba: evento repetido, no se duplica
```

imagen 17. Duplicado y manejo del mismo.

Revisando el dato viejo y como la caché puede llegar a jugarnos una mala pasada haremos un cambio de saldo en la cuenta de Ana y revisaremos constantemente el saldo de su cuenta para ver hasta que punto se deja de revisar la copia en Redis (caché) y se actualiza el saldo actual con Postgres:

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/002180000000000001
{"clabe":"002180000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-04T20:35:11.40927357
4Z"}
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T db psql -U mexibanco -d cuenta -c "update cuenta set saldo = 5 where clabe
= '002180000000000001'"
UPDATE 1

jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 8); do
  docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
  curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
  sleep 5
done
```

imagen 18. Cambio de saldo y consulta del mismo.

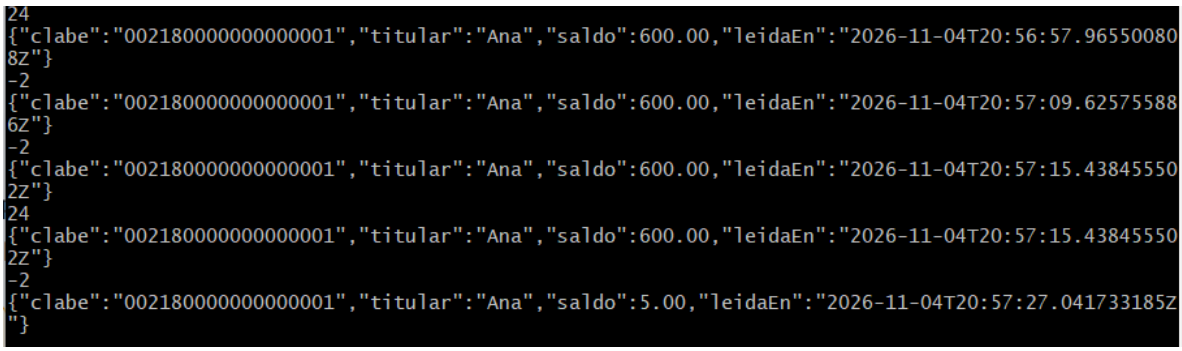


imagen 19. Actualización de saldo.

Cómo se puede ver en la imagen 19, el saldo de Ana se actualiza a mitad de las consultas, esto ocurre porque la copia muere y se revisa el saldo directamente desde Postgres, actualizando así el saldo que tiene en realidad Ana y mostrándolo adecuadamente.

Para seguir las pruebas y que pasaría con el saldo de la caché en una transferencia, haremos un movimiento de prueba. Con ello veremos que pasa con el saldo y si se efectúa la transferencia con el saldo de la caché.

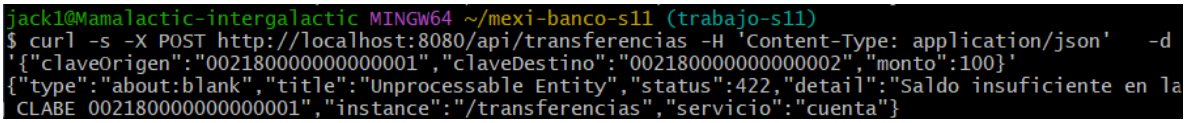


imagen 20. Transferencia de prueba con saldo de la caché.

Lo que nos arroja el sistema es que no se pudo hacer la transferencia debido a un saldo insuficiente, esto tiene que ver con la modificación que hicimos en el código, ya que el método consultar no utiliza la caché (o Redis), en su lugar utiliza una consulta directa a Postgres para tener siempre el saldo en tiempo real y evitar que la caché cause problemas.

6. **Preguntas de cierre**

1. **¿Por qué MongoDB para los avisos y no para el saldo? Usa lo que viste: la forma de cada aviso, cómo se leen y qué tendría que garantizar el saldo.**

Porque los avisos de notificación tienen una estructura basada en plantillas, como el SMS, con un teléfono y texto, el correo una dirección, asunto y cuerpo; y un push un token, título y cuerpo. Por lo que en una base de datos relacional como PostgreSQL, el modelado de esto requeriría una tabla con múltiples columnas vacías para canales que no aplican, o tablas separadas para cada canal. Sin embargo, MongoDB al ser tipo NoSQL permite almacenar este tipo de avisos flexibles, facilitando el almacenamiento y la escritura.

Y en el caso del saldo, este requiere la mayor garantía de consistencia usando ACID, una característica propia de una base de datos relacional como lo es Postgres. Además de que el saldo se modifcia constantemente y sobre él se toman decisiones criticas, como la autorización de cargos o transferencias, por lo que no puede depender de caché o almacenamiento NoSQL.

2. **¿Qué hit ratio mediste en A y en B, y por qué cambia? Con tus números de latencia y de lecturas a Postgres: ¿Qué compraste con la caché, velocidad o capacidad?**

Prueba A:

98 hits y 2 misses. Esto demuestra que casi todas las peticiones se resolvieron directamente desde Redis sin afectar directamente a la base de datos.

Prueba B:

99 hits y 21 misses

Cambia B con respecto a A porque cada transferencia ejecuta un @CacheEvict y hace Delete sobre las cuentas involucradas, tanto el origen como el destinatario, lo que hace que se invalide la copia de Redis. Haaciendo que la lectura siguiente a una transferencia genere un miss, volviendo a revisar el dato en Postgres y rellenando la caché una vez más con un nuevo SET.

Lo que se compró fue la velocidad, notándose demasiado en la reducción de latencia y el tiempo de operaciones, a la vez que se obtuvo capacidad para Postgres, liberando la carga de consultas que tenía que procesar repetitivamente.

3. **¿Qué pasaría si la autorización del cargo leyera el saldo de la caché? Usa lo que viste en la parte 4.**

El sistema autorizaría con datos obsoletos. Esto tiene que ver justo con la última parte del lab. En donde revisamos que la consulta fuera de la Caché evita este problema. Pero suponiendo que se permitiera hacer movimientos con un saldo desactualizado, esto permitiría sobregiros, doble gasto o inconsistencias en el sistema, donde la aplicación creería que hay fondos disponibles cuando en realidad no los hay.

4. **Al aplicar un movimiento, ¿Por qué se hace DEL de la copia y no SET con el saldo nuevo? ¿Y por qué después del commit y no antes?**

Según el patrón de Cache-Aside, invalidar la caché con un DEL previene condiciones de carrera y estados inconsistentes si ocurren múltiples transacciones afectando al saldo. Al borrar la caché se fuerza a que la siguiente consulta genere un miss y se lea el saldo nuevamente desde Postgres y se actualicen los datos en la caché.

Se hace después del commit porque de esta forma evitamos que a media transferencia alguien revise el saldo y generé una copia en Redis, guardando un saldo equivocado en la caché por unos segundos. Teniendo un dato obsoleto en Redis. Así que al hacer un commit después, nos aseguramos de que el saldo se mantenga en orden.